SLUB صف‌های هر پردازنده را که تخصیص‌گر SLAB برای اشیاء هر یک از کَش‌ها نگه می‌دارد، حذف می‌کند. برای سیستم‌هایی که تعداد پردازنده‌هایشان زیاد است، میزان حافظه‌ای که برای این صف‌ها تخصیص می‌یابد قابل توجه است. بنابراین، هر چه تعداد پردازنده‌های روی یک سیستم افزایش می‌یابد، SLUB کارایی بهتری فراهم می‌کند.

۱۰-۹ ملاحظات دیگر

تصمیمات مهمی که در رابطه با سیستم صفحه‌بندی داریم گزینش یک الگوریتم جایگزینی و اتخاذ یک سیاست تخصیص می‌باشد که پیشتر در همین فصل مورد بحث قرار دادیم. در همین راستا، ملاحظات فراوان دیگری هم هستند که برخی از آنها را در اینجا مورد بحث قرار می‌دهیم.

۱۰-۹-۱ پیش‌صفحه‌بندی

ویژگی بارز صفحه‌بندی نیازی محض، تعداد فراوان خطاهای صفحه‌ای است که در زمان شروع یک پردازه رخ می‌دهند. این وضعیت ناشی از تلاشی است که در آوردن اولین همسایگی به درون حافظه صورت می‌گیرد. **پیش‌صفحه‌بندی**[1] اقدامی است که می‌خواهد از خطای زیاد صفحات در صفحه‌بندی آغازین پردازه پیشگیری نماید. راهبرد آن است که برخی از—یا تمامی—صفحاتی را که لازم هستند در یک زمان به‌حافظه بیاوریم.

برای مثال، در سیستمی که از مدل مجموعه‌ی کاری استفاده می‌کند، می‌توانیم با هر یک از پردازه‌ها لیستی از صفحات مجموعه‌ی کاری آن را نگه بداریم. اگر به ناچار پردازه‌ای را معلق کردیم (به دلیل نبود قاب‌های آزاد)، مجموعه‌ی کاری آن پردازه را به خاطر می‌سپاریم. لحظه‌ای که قرار است پردازه از سر گرفته شود (به واسطه‌ی تکمیل I/O یا وجود قاب‌های آزاد کافی)، باز هم کل مجموعه‌ی کاری پردازه را به طور خودکار پیش از شروع مجدد پردازه به حافظه می‌آوریم.

پیش‌صفحه‌بندی در موارد خاصی می‌تواند سودآور باشد. سؤال ساده‌ای که مطرح می‌شود این است که آیا هزینه‌ی پیش‌صفحه‌بندی از هزینه‌ی سرویس خطای صفحات متناظر کمتر است یا بیشتر. ممکن است حالتی پیش بیاید که صفحات زیادی با پیش‌صفحه‌بندی به حافظه آورده شوند که استفاده هم نخواهند داشت.

فرض کنید تعداد s صفحه پیش‌صفحه‌بندی می‌شوند و α ($۰ \leq \alpha \leq ۱$) کسری از این صفحات s است که به واقع استفاده خواهند داشت. این سؤال مطرح می‌شود که آیا هزینه‌ی صرفه‌جویی $s \times \alpha$ خطای صفحه از هزینه‌ی پیش‌صفحه‌بندی $(۱ - \alpha) \times s$ صفحه‌ی غیر ضروری بیشتر است یا کمتر. اگر α نزدیک ۰ باشد، پیش‌صفحه‌بندی می‌بازد؛ اگر α نزدیک ۱ باشد، پیش‌صفحه‌بندی می‌برد.

توجه داشته باشید که پیش‌صفحه‌بندی یک برنامه‌ی اجرایی ممکن است دشوار باشد، چرا که دقیقاً واضح نیست که کدام صفحات بایستی به داخل حافظه آورده شوند. پیش‌صفحه‌بندی یک فایل می‌تواند قابل پیش‌بینی‌تر باشد، چون فایل‌ها غالباً به‌ترتیب دست‌یابی می‌شوند. فراخوانی

1. Prepaging

سیستمی readahead() لینوکس محتویات یک فایل را به داخل حافظه پیش‌واکشی می‌کند، طوری که دسترسی‌های بعدی فایل از حافظه‌ی اصلی انجام می‌شوند.

۱۰-۹-۲ اندازه‌ی صفحه

طراحان یک سیستم‌عامل به ندرت نگرانی انتخاب اندازه‌ی صفحات یک ماشین موجود را خواهند داشت. اما، زمانی که ماشین‌های جدیدی طراحی می‌شوند، بایستی یک تصمیم در مورد بهترین اندازه‌ی صفحه اتخاذ شود. همان‌گونه که حدس می‌زنید اندازه‌ی منفردی که برای صفحه بهترین باشد وجود ندارد. بلکه، مجموعه‌ای از عوامل وجود دارد که اندازه‌های مختلف را مطرح می‌کند. به طور کلی، اندازه‌ی صفحات با توان‌هایی از ۲، عموماً در بازه‌ی ۴۰۹۶ (2^{12}) تا ۴۱۹۴۳۰۴ (2^{22}) بایت تغییر می‌کند.

چگونه یک اندازه‌ی صفحه را انتخاب کنیم؟ یکی از دغدغه‌ها اندازه‌ی جدول صفحه است. در یک فضای حافظه‌ی مجازی، هرچه اندازه‌ی صفحه کاهش یابد، تعداد صفحات و از این رو، اندازه‌ی جدول صفحه افزایش پیدا می‌کند. برای مثال، با یک حافظه‌ی مجازی ۴ مگابایتی (2^{22} بایت) می‌توان ۴۰۹۶ صفحه‌ی ۱۰۲۴ بایتی، اما تنها ۵۱۲ صفحه‌ی ۸۱۹۲ بایتی داشت. چون هر پردازه‌ای که اجرا می‌شود بایستی کپی جدول صفحه‌ی خاص خود را داشته باشد، اندازه‌ی بزرگ صفحات مطلوب‌تر است.

با این وجود، بهره‌وری حافظه با صفحات کوچکتر بهتر است. اگر حافظه‌ای که برای پردازه‌ای تخصیص می‌یابد با شروع از مکان 00000 تداوم یابد، احتمالاً آخرین صفحه‌اش را به طور کامل پر نخواهد کرد. بنابراین، بخشی از آخرین صفحه‌ای که تخصیص می‌یابد (چون واحد تخصیص ما صفحات هستند) بلااستفاده می‌ماند (ایجاد پارگی داخلی). با فرض این که، اندازه‌ی پردازه و اندازه‌ی صفحه مستقل از هم هستند، انتظار داریم که به طور متوسط، نیمی از آخرین صفحه‌ای که به پردازه تخصیص می‌یابد هدر برود. این اتلاف برای صفحات ۵۱۲ بایتی فقط ۲۵۶ بایت است، اما برای صفحات ۸۱۹۲ بایتی، برابر ۴۰۹۶ بایت خواهد بود. لذا، اندازه‌ی صفحات کوچکتر را به خاطر کمینه‌سازی پارگی داخلی ترجیح می‌دهیم.

مسأله‌ی دیگر، مدت زمانی است که برای خواندن یا نوشتن یک صفحه لازم می‌شود. همان‌گونه که در بخش ۱۱-۱ خواهید دید، موقعی که دستگاه ذخیره‌سازی HDD می‌باشد، زمان I/O مرکب از زمان استوانه‌یابی، دیرکرد دورانی و زمان انتقال می‌باشد. زمان انتقال با میزان انتقال متناسب است (یعنی، اندازه‌ی صفحه)—واقعیتی که به‌نظر برای اندازه‌ی صفحه‌ی کوچک‌تر بهتر است. با این وجود، زمان دیرکرد دورانی و استوانه‌یابی معمولاً به زمان انتقال می‌چربند. با یک نرخ انتقال ۵۰ مگابایت در ثانیه، انتقال ۵۱۲ بایت تنها ۰/۰۱ میلی‌ثانیه زمان می‌برد. به رغم این، شاید در این انتقال به دیرکرد دورانی ۳ میلی‌ثانیه‌ای و استوانه‌یابی ۵ میلی‌ثانیه‌ای هم نیاز داشته باشیم. بنابراین، از مجموع زمان I/O (۸/۰۱ میلی‌ثانیه)، فقط حدود ۰/۱ درصد آن سهم انتقال واقعی است. اگر اندازه‌ی صفحات دو برابر شوند زمان I/O فقط به ۸/۰۲ میلی‌ثانیه افزایش می‌یابد. خواندن یک تک‌صفحه با اندازه‌ی ۱۰۲۴ بایت به ۸/۰۲ میلی‌ثانیه زمان نیاز دارد، ولی خواندن همین

میزان حافظه ولی با دو صفحه‌ی ۵۱۲ بایتی به ۱۶/۰۲ میلی‌ثانیه زمان نیازمند است. بنابراین، برای کمینه‌سازی زمان I/O مطلوب‌تر است که موافق اندازه‌ی صفحات بزرگ‌تر باشیم.

هرچند که کل I/O با اندازه‌ی صفحات کوچکتر می‌تواند کاهش پیدا کند، چرا که همسایگی بهبود پیدا خواهد کرد. اندازه‌ی صفحات کوچکتر باعث می‌شوند که هر یک از صفحات به صورت واقعی‌تر با همسایگی برنامه انطباق داشته باشند. برای مثال، پردازه‌ای با اندازه‌ی ۲۰۰ کیلوبایت را در نظر بگیرید که به واقع تنها نیمی از آن (۱۰۰ کیلوبایت) در زمان اجرا مورد استفاده قرار خواهد گرفت. اگر تنها یک صفحه‌ی بزرگ داشته باشیم، بایستی کل پردازه را به صفحه بیاوریم که در مجموع به انتقال و تخصیص ۲۰۰ کیلوبایت نیاز خواهیم داشت. اما اگر صفحات ما ۱ بایتی باشند، در این صورت فقط ۱۰۰ کیلوبایتی را که واقعاً استفاده خواهد شد به حافظه می‌آوریم که با این کار، تنها به انتقال و تخصیص ۱۰۰ کیلوبایت نیاز داریم. پس، با اندازه‌ی صفحات کوچکتر به **تفکیک** بهتری دست می‌یابیم و می‌توانیم آن بخش از حافظه را که واقعاً لازم می‌شود جدا کنیم. با اندازه‌ی صفحات بزرگ‌تر، نه تنها باید آنچه را که لازم است تخصیص و انتقال بدهیم، بلکه بایستی هر چیز دیگری را که در داخل صفحه وجود دارد، فارغ از این که مورد نیاز است یا خیر انتقال دهیم. بنابراین، کوچکی اندازه‌ی صفحات موجب I/O کمتر می‌شود و مجموع حافظه‌ی تخصیصی نیز کاهش می‌یابد.

اما آیا توجه دارید که با اندازه‌ی صفحات ۱ بایتی، برای هر بایت یک خطای صفحه خواهیم داشت؟ پردازه‌ی ۲۰۰ کیلوبایتی که تنها از نصف حافظه‌اش استفاده می‌کرد، با صفحات ۲۰۰ کیلوبایتی فقط یک خطای صفحه تولید می‌کند، اما با اندازه‌ی صفحات ۱ بایتی به تعداد ۱۰۲۴۰۰ خطای صفحه تولید خواهد نمود. هر خطای صفحه‌ای که اتفاق می‌افتد سربار قابل توجه پردازش وقفه، ذخیره‌سازی ثبات‌ها، جایگزینی صفحه، صف دستگاه صفحه‌بندی و به روزکردن جدول‌ها را به دنبال دارد. برای کمینه‌سازی تعداد خطاهای صفحه به اندازه‌ی بزرگ صفحات نیاز داریم.

عوامل دیگری نیز هستند (نظیر رابطه‌ی میان اندازه‌ی صفحه و اندازه‌ی قطاع روی دستگاه صفحه‌بندی) که بایستی منظور شوند. برای این مسأله، بهترین جواب وجود ندارد. همان‌گونه که دیده‌ایم، برخی عوامل (پارگی داخلی، همسایگی) روی اندازه‌ی کوچک صفحات توافق دارند، حال آنکه عوامل دیگری (اندازه‌ی جدول، زمان I/O) نیز موافق اندازه‌ی بزرگ صفحات هستند. با این وجود، یک گرایش تاریخی، حتی در سیستم‌های موبایل بر روی اندازه‌ی صفحات بزرگ‌تر وجود دارد. در حقیقت، در زمان اولین ویرایش *مفاهیم سیستم عامل* (۱۹۸۳)، از ۴۰۹۶ بایت به عنوان کران بالای اندازه‌ی صفحات استفاده می‌شد که این مقدار در سال ۱۹۹۰ به عنوان رایج‌ترین اندازه‌ی صفحه به حساب می‌آمد. اما، همان‌گونه که در بخش بعدی خواهید دید، اکنون سیستم‌های مدرنی هستند که از اندازه‌ی صفحات بزرگ‌تری نیز استفاده می‌کنند.

۱۰-۹-۳ حجم پوششی TLB

در فصل ۹، **ضریب توفیق** TLB را معرفی کردیم. به خاطر دارید که ضریب توفیق TLB درصد دفعاتی بود که می‌توانستیم آدرس‌های مجازی را به جای جدول صفحه با استفاده از TLB به آدرس فیزیکی ترجمه کنیم. روشن است که ضریب توفیق به تعداد واردهای TLB مربوط

می‌شود و راه افزایش ضریب توفیق، افزایش تعداد واردەها می‌باشد. اما، این کار ارزان نیست، چون برای ساخت TLB از حافظه‌ی انجمنی استفاده می‌شود که هم گران است و نیز حرص انرژی دارد.

مرتبط با ضریب توفیق یک معیار مشابه وجود دارد: **حجم پوششیTLB**[1]. حجم پوششی TLB به میزان حافظه‌ای مربوط می‌شود که از TLB قابل دسترسی است و به سادگی با حاصلضرب تعداد واردەهای TLB در اندازه‌ی صفحه بدست می‌آید. به صورت ایده‌آل، مجموعه‌ی کاری پردازه در TLB ذخیره می‌شود. اگر غیر از این باشد، پردازه زمان قابل‌توجهی را برای دستیابی مراجعات حافظه به جای TLB صرف جدول صفحه خواهد نمود. اگر تعداد واردەهای TLB را دو برابر کنیم، حجم پوششی TLB را دو برابر کرده‌ایم. با این وجود، برای برخی برنامه‌های کاربردی که عطش حافظه دارند باز هم امکان دارد که در ذخیره‌سازی مجموعه‌ی کاری چنین کاربردهایی با کمبود فضا مواجه شویم.

رویکرد دیگری که برای افزایش حجم پوششی TLB می‌تواند مطرح شود افزایش اندازه‌ی صفحه و یا استفاده از اندازه‌ی صفحات چندگانه می‌باشد. اگر اندازه‌ی صفحات را—برفرض از ۴ کیلوبایت به ۱۶ کیلوبایت—افزایش بدهیم، حجم پوششی TLB را چهار برابر می‌کنیم. با این وجود، این کار در مورد آن دسته از برنامه‌های کاربردی که نیازی به اندازه‌ی بزرگ صفحات ندارند منجر به افزایش پارگی می‌شود. به روشی دیگر، اکثر معماری‌ها بیشتر از یک اندازه‌ی صفحه را پشتیبانی می‌کنند، و یک سیستم‌عامل می‌تواند پیکربندی شود تا از این پشتیبانی بهره ببرد. برای مثال، اندازه‌ی صفحه‌ی پیش‌فرض در سیستم‌های لینوکس ۴ کیلوبایت است؛ با این وجود، لینوکس **صفحات بسیار بزرگ** را نیز فراهم می‌کند، امکانی که یک ناحیه از حافظه‌ی فیزیکی را مشخص می‌کند که ممکن است صفحات بزرگ‌تری (برای مثال، ۲ مگابایت) استفاده شوند.

از بخش ۹-۷ به‌یاد بیاورید که معماری ARMv8، صفحات و نواحی با اندازه‌های متفاوت را پشتیبانی می‌کند. به‌علاوه، هر واردەی TLB در ARMv8 شامل یک **بیت همجوار**[2] است. اگر این بیت برای یک واردەی خاصی از TLB انتخاب شود، آن وارده به بلاک‌های همجواری (مجاور) از حافظه نگاشته می‌شود. بلاک‌های همجوار می‌توانند در یک واردەی منفرد TLB به سه ترتیب ممکن نگاشته شوند، با این کار حجم پوششی TLB افزایش پیدا می‌کند:

۱- واردەی TLB ۶۴ کیلوبایتی مرکب از بلاک‌های مجاور ۴ × ۱۶ کیلوبایتی.
۲- واردەی TLB ۱ گیگابایتی مرکب از بلاک‌های مجاور ۳۲ × ۳۲ مگابایتی.
۳- واردەی TLB ۲ مگابایتی مرکب از بلاک‌های مجاور ۶۴ × ۳۲ کیلوبایتی، یا بلاک‌های مجاور ۱۶ × ۱۲۸ کیلوبایتی.

تدارک پشتیبانی برای اندازه‌های متعدد صفحات ممکن است لازم داشته باشند که سیستم‌عامل—در عوض سخت‌افزار—TLB را مدیریت کند. برای مثال، یکی از فیلدها در یک واردەی TLB بایستی اندازه‌ی قاب صفحه‌ی متناظر وارده را نشان بدهد—یا، در مورد

1. TLB reach
2. Contiguous bit

معماری‌های ARM، بایستی نشان دهد که وارده به یک بلاک همجواری از حافظه رجوع می‌کند. مدیریت TLB در نرم‌افزار و نه سخت‌افزار، باعث هزینه در کارایی می‌شود. با این وجود، ضریب توفیق افزایش‌یافته و حجم پوششی TLB، این هزینه‌های کارایی را جبران می‌کنند.

۱۰-۹-۴ جدول صفحات وارونه

بخش ۹-۴-۳، مفهوم جدول صفحه‌ی وارونه را معرفی کرد. هدف از این شکل مدیریت صفحه، کاهش حجم حافظه‌ی فیزیکی مورد نیاز در پیگیری ترجمه‌ی آدرس‌های مجازی به فیزیکی می‌باشد. این صرفه‌جویی را با ایجاد جدولی که تنها یک وارده برای هر یک از صفحات فیزیکی دارد انجام می‌دهیم که با زوج <process-id, page-number> ایندکس می‌شود.

چون جدول صفحات وارونه اطلاعات مربوط به این که، کدام صفحه‌ی حافظه‌ی مجازی در هر یک از قاب‌های فیزیکی ذخیره می‌شود را نگه می‌دارند، میزان حافظه‌ی فیزیکی مورد نیاز ذخیره‌سازی این اطلاعات را کاهش می‌دهند. با این وجود، جدول صفحه‌ی وارونه دیگر فاقد اطلاعات کامل مربوط به فضای آدرس منطقی پردازه می‌باشد و این اطلاعات در زمان رجوع به صفحه‌ای که فعلاً در حافظه نیست لازم خواهند شد. صفحه‌بندی نیازی برای پردازش خطاهای صفحه به این اطلاعات نیازمند است. برای مهیاکردن این اطلاعات لازم است که یک جدول صفحه‌ی خارجی (یکی برای هر پردازه) نگه‌داری شود. هر یک از این جدول‌ها مشابه جدول صفحات سابق هر یک از پردازه‌هاست و اطلاعات مربوط به مکان قرارگیری هر یک از صفحات مجازی را شامل می‌شود.

ولی آیا جدول صفحات خارجی، کارآمدی و سودمندی جدول صفحات وارونه را کم‌رنگ نمی‌سازند؟ چون این جدول‌ها تنها در زمان رخداد خطای صفحه رجوع می‌شوند، نیازی نیست که به سرعت فراهم شوند. در عوض، آنها خود در مواقع نیاز به داخل و خارج از حافظه صفحه‌بندی می‌شوند. متأسفانه، اکنون وقوع یک خطای صفحه می‌تواند باعث شود تا مدیر حافظه‌ی مجازی برای صفحه‌بندی جدول صفحه‌ی خارجی نیز که برای یافتن مکان صفحه‌ی مجازی روی ذخیره‌ی پشتیبان لازم می‌شود، خطای صفحه‌ی دیگری تولید کند. این مورد خاص، مستلزم دقت راه‌اندازی و رسیدگی هسته و یک تأخیر در پردازش مراجعه به صفحه است.

۱۰-۹-۵ ساختار برنامه

صفحه‌بندی نیازی طوری طراحی می‌شود که برای برنامه‌ی کاربر شفاف باشد. در بسیاری از موارد، کاربر کاملاً از طبیعت صفحه‌بندی حافظه بی‌اطلاع است. با این وجود، شرایط دیگری هم وجود دارد که آگاهی کاربر (یا کامپایلر) از صفحه‌بندی نیازی زیرین می‌تواند کارایی سیستم را بهبود ببخشد.

یک مثال ساختگی، اما روشن‌کننده را خواهیم دید. فرض کنید اندازه‌ی صفحات ۱۲۸ کلمه هستند. برنامه‌ای به زبان C را که کار آن ۰ کردن هر یک از عناصر آرایه‌ی دو بعدی ۱۲۸ در ۱۲۸ می‌باشد در نظر بگیرید. کد نوعی برنامه به صورت زیر است:

```
int i, j;
```

```
int data[128][128];
for (j = 0; j < 128; j++)
        for (i = 0; i < 128; i++)
                data[i][j] = 0;
```

توجه داشته باشید که آرایه در حافظه به صورت سطری ذخیره می‌شود؛ یعنی، آرایه به صورت data[0][0]، data[0][1]، ...، data[0][127]، data[1][0]، data[1][1]، ...، data[127][127] ذخیره می‌شود. با صفحات ۱۲۸ کلمه‌ای، هر سطر در یک صفحه قرار می‌گیرد. بنابراین، این کد یک کلمه را در هر صفحه صفر کرده، سپس سراغ کلمه‌ی بعدی از صفحه‌ی دیگر می‌رود و الی آخر. اگر سیستم‌عامل به کل برنامه کمتر از ۱۲۸ قاب تخصیص بدهد، در این صورت اجرای این کد، به تعداد 16,384 = 128 × 128 خطای صفحه خواهد داشت. در مقایسه، اگر کد را به صورت زیر تغییر بدهیم:

```
int i, j;
int data[128][128];
for (i = 0; i < 128; i++)
        for (j = 0; j < 128; j++)
                data[i][j] = 0;
```

این کد پیش از این که صفحه‌ی بعدی را شروع کند تمامی کلمات صفحه‌ی جاری را صفر می‌کند، لذا تعداد خطای صفحات را به ۱۲۸ کاهش می‌دهد.

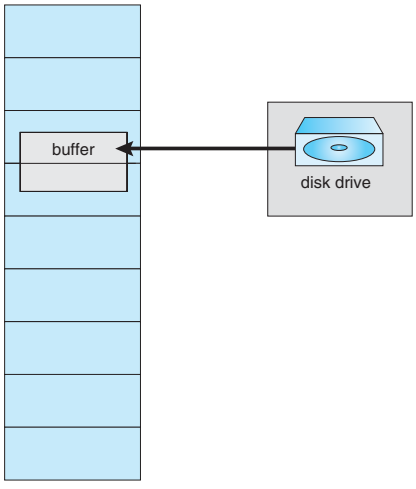
دقت در انتخاب ساختمان‌داده‌ها و ساختارهای برنامه‌نویسی می‌تواند همسایگی را افزایش داده و از این رو، نرخ خطای صفحه و تعداد صفحات مجموعه‌ی کاری را پایین بیاورد. برای مثال، پشته همسایگی خوبی دارد، چون دسترسی آن همیشه از بالا صورت می‌گیرد. در مقایسه، جدول درهم طوری طراحی می‌شود تا مراجعات را پراکنده سازد؛ لذا، همسایگی بدی تولید می‌کند. البته، همسایگی مراجعات فقط یکی از معیارهای سودمندی است که بایستی در استفاده از ساختمان‌داده‌ها مد نظر قرار بگیرد. عوامل دیگری که در این رابطه وزن بالایی هم دارند، عبارتند از سرعت جستجو، تعداد کل مراجعات حافظه و تعداد کل صفحاتی که درگیر می‌شوند.

در گامی دیگر، کامپایلر و بارکننده می‌توانند تأثیر به سزایی بر روی صفحه‌بندی داشته باشند. تفکیک کد و داده‌ها و تولید کد چندمدخلی به این معنی است که صفحات کد، فقط‌خواندنی بوده و از این رو هرگز تغییر پیدا نخواهند کرد. صفحات تمیزی که برای جایگزینی انتخاب می‌شوند الزامی به نوشتن در دیسک ندارند. بارکننده می‌تواند از قراردادن روال‌ها در مرز صفحات اجتناب کرده و هر یک از روال‌ها را به طور کامل در یک صفحه نگه بدارد. روال‌هایی که همدیگر را به کرات فراخوانی می‌کنند می‌توانند در صفحه‌ای یکسان قرار داده شوند. این بسته‌بندی، گونه‌ای از مسأله‌ی bin-packing تحقیق در عملیات است: سعی کن قطعات بار با اندازه‌ی متغیر را طوری در صفحات با اندازه‌ی ثابت بسته‌بندی کنی که مراجعات مابین صفحات کمینه گردد. این روش مخصوصاً در رابطه با اندازه‌ی بزرگ صفحات پرفایده خواهد بود.

۱۰-۹-۶ قفل I/O در حافظه و قفل صفحه

زمانی که از صفحه‌بندی نیازی استفاده می‌کنیم، بعضاً لازم می‌شود که برخی صفحات داخل حافظه **قفل** شوند. یکی از این شرایط زمانی است که I/O به و یا از حافظه‌ی (مجازی) کاربر صورت می‌گیرد. در اغلب موارد، I/O توسط پردازنده‌ی مجزایی پیاده‌سازی می‌شود. برای مثال، در حالت کلی، تعداد بایت‌های انتقالی و آدرس حافظه‌ی بافر به کنترلر دستگاه ذخیره‌سازی USB داده می‌شود (شکل ۱۰-۲۸). لحظه‌ای که انتقال کامل شد، پردازنده وقفه دریافت می‌کند.

بایستی تضمین کنیم که دنباله‌ی رویدادهای زیر اتفاق نمی‌افتند: پردازه‌ای یک درخواست I/O صادر می‌کند و در صف مربوط به دستگاه I/O قرار داده می‌شود. در این میان، پردازنده در اختیار پردازه‌های دیگری قرار می‌گیرد. این پردازه‌ها خطای صفحه می‌دهند و یکی از آنها، با استفاده از الگوریتم جایگزینی سراسری، صفحه‌ای را که شامل بافر پردازه‌ی منتظر است جایگزین می‌کند. صفحات در دیسک نوشته می‌شوند. مدت زمانی سپری می‌شود، زمانی که درخواست I/O در صف به پیش رفته و به سر صف دستگاه می‌رسد، I/O به آدرسی که مشخص شده بود انجام می‌شود. اما، اکنون این قاب برای صفحه‌ای متفاوت که به پردازه‌ی دیگری هم تعلق دارد، استفاده می‌شود.



شکل ۱۰-۲۸ دلیل این که چرا قاب‌های مورد استفاده برای I/O بایستی داخل حافظه قرار داشته باشند.

این مسأله دو راه‌حل رایج دارد. یک راه‌حل آن است که عملیات I/O را هرگز در حافظه‌ی کاربر اجرا نکنیم. به جای آن، داده‌ها همواره بین حافظه‌ی سیستم و حافظه‌ی کاربر کپی می‌شوند. عملیات I/O فقط میان حافظه‌ی سیستم و دستگاه I/O صورت می‌گیرد. بنابراین، برای نوشتن یک بلاک در نوار، نخست بلاک را به حافظه‌ی سیستم کپی کرده و سپس آن را روی نوار می‌نویسیم. این کپی اضافی می‌تواند سربار زیاد و نامقبولی داشته باشد.

راه‌حل دیگر آن است که امکان قفل صفحات داخل حافظه را فراهم کنیم. در اینجا، برای هر یک از قاب‌ها یک بیت قفل منظور می‌شود. اگر قابی قفل شده باشد، نمی‌توان آن را برای جایگزینی انتخاب کرد. با این روش، برای نوشتن یک بلاک در دیسک، ابتدا صفحات مربوط به بلاک را در حافظه قفل می‌کنیم. سپس، سیستم می‌تواند عملیات خود را طبق معمول ادامه بدهد. صفحات قفل‌شده نمی‌توانند جایگزین شوند. لحظه‌ای که I/O تکمیل می‌شود، قفل صفحات باز می‌گردند.

قفل‌های بیت در شرایط مختلفی استفاده می‌شوند. زیاد اتفاق می افتد که قسمتی یا تمامی هسته‌ی سیستم‌عامل در داخل حافظه قفل می‌شود. بسیاری از سیستم‌عامل‌ها تحمل خطای صفحه‌ای را ندارند که توسط هسته یا یک ماژول خاص هسته، از جمله ماژولی که مدیریت حافظه را انجام می‌دهد، حادث شود. پردازه‌های کاربر هم ممکن است بخواهند صفحاتی را در داخل حافظه قفل کنند. یک پردازه‌ی پایگاه‌داده ممکن است بخواهد بخشی از حافظه را مدیریت کند، برای مثال، جابجایی بلاک‌ها میان حافظه‌ی ثانویه و حافظه که خود پایگاه‌داده، بهترین دانش چگونگی استفاده از این داده‌ها را داراست. این قبیل **میخکوبی** صفحات در حافظه کاملاً رایج است و اکثر سیستم‌عامل‌ها یک فراخوانی سیستمی دارند که به یک برنامه اجازه می‌دهد تا درخواستی برای میخکوبی یک ناحیه از فضای آدرس منطقی خودش را صادر کند. توجه داشته باشید که این امکان ممکن است مورد سوءاستفاده واقع شده و روی الگوریتم‌های مدیریت حافظه فشار بیاورد. بنابراین، در بیشتر موارد، یک برنامه بایستی امتیازهای خاصی بدست آورد تا بتواند چنین درخواستی را بدهد.

استفاده‌ی دیگر قفل بیت در جایگزینی معمول صفحه می‌باشد. توالی رویدادهای زیر را در نظر بگیرید: پردازه‌ای با الویت پایین خطای صفحه می‌دهد. سیستم صفحه‌بندی، صفحه‌ی مورد نیاز پردازه را با جایگزینی قابی به داخل حافظه می‌خواند. در ادامه، پردازه‌ی با الویت پایین وارد صف آماده می‌شود و به انتظار پردازنده می‌ماند. چون الویت پایینی دارد اعزام‌کننده آن را به این زودی برای اجرا انتخاب نخواهد کرد. در حالی که پردازه‌ی الویت پایین منتظر است، پردازه‌ای با الویت بالا خطای صفحه می‌دهد. سیستم صفحه‌بندی در جستجو به دنبال صفحه‌ای برای جایگزینی، صفحه‌ای را می‌یابد که در حافظه قرار دارد که نه مراجعه‌ای داشته و نه تغییر یافته است: این همان صفحه‌ای است که پردازه‌ی الویت پایین چند لحظه قبل به حافظه آورده بود. این صفحه به نظر جایگزین کاملی است: تمیز است و نیازی به نوشتن در دیسک را هم ندارد و یقیناً برای مدت طولانی هم استفاده نخواهد داشت.

این که پردازه‌ای با الویت بالا بایستی بتواند صفحه‌ای از پردازه‌ی با الویت پایین را برای جایگزینی انتخاب کند، اتخاذ یک سیاست است. در هر صورت، با این کار، پردازه‌ای را که الویت پایینی دارد به نفع پردازه‌ای که الویت بالاتری دارد به تأخیر می‌اندازیم. با این وجود، تلاشی را که پردازه‌ی با الویت پایین در آوردن صفحه به داخل حافظه صرف کرده بود هدر می‌دهیم. اگر تصمیم بگیریم که مانع جایگزینی صفحه‌ای شویم که به تازگی وارد حافظه شده است تا حداقل یکبار هم که شده مورد استفاده قرار بگیرد، در این صورت برای پیاده‌سازی این راهکار می‌توانیم از بیت قفل استفاده کنیم. لحظه‌ای که صفحه‌ای برای جایگزینی انتخاب می‌شود بیت قفل آن یک می‌شود و مادامی که پردازه‌ی خطادهنده دوباره برای اجرا اعزام نشده است قفل می‌ماند.